Université Hassan 1er — Faculté des Sciences et Techniques

Département Informatique

Système de gestion d'une pharmacie

Projet de programmation Java

Encadrant Pr. Said El Kafhali

Réalisé par Oussama Tarchi
Ali Zouine
Taha Rajel
Abdessamad Lyamani

Année universitaire 2025/2026

Table des matières

1 Introduction générale **4**

2 Analyse du besoin **5**

2.1 Présentation du sujet 5

2.2 Besoins fonctionnels 5

2.3 Besoins non fonctionnels 6

3 Conception du système **7**

3.1 Architecture du projet 7

3.2 Structure des packages 7

3.3 Classes principales du modèle 8

3.4 Diagramme de cas d'utilisation 8

3.5 Diagramme de classes 9

3.6 Diagramme entité-relation 9

4 Base de données **10**

4.1 Présentation de la base 10

4.2 Tables et relations 10

4.3 Clés primaires et étrangères 11

4.4 Correspondance noms SQL et DAO 12

5 Réalisation technique **13**

5.1 Interface graphique : Java Swing 13

5.2 Accès aux données : JDBC 13

5.3 Couche DAO 15

5.4 Couche Contrôleur 15

5.5 Génération des factures texte 16

5.6 Point d'entrée 16

6 Fonctionnement de l'application **18**

6.1 Gestion des médicaments 18

6.2 Gestion des clients 19

6.3 Gestion des ventes 19

6.4 Génération automatique de facture 20

6.5 Commande fournisseur et réception 21

6.6 Suivi du stock et alertes 22

6.7 Analyse avec le dashboard 23

7 Problèmes rencontrés **25**

7.1 Modélisation des ventes et des lignes de vente 25

7.2 Intégration de l'ordonnance dans la vente 25

7.3 Ajout d'une ligne avant création de la vente 26

7.4 Association tardive de l'ordonnance à la vente 26

7.5 Génération et stockage des factures texte 26

7.6 Séparation des fichiers DAO et Controller 27

8 Tests et validation **28**

8.1 Test de gestion des clients 28

8.1.1 Ajout d'un client 28

8.1.2 Modification des informations 28

8.1.3 Suppression d'un client 29

8.2 Test de gestion des médicaments 29

8.2.1 Ajout d'un médicament 29

8.2.2 Recherche par nom 30

8.3 Test de gestion des ventes 31

8.3.1 Création d'une vente 31

8.3.2 Ajout de plusieurs médicaments dans une même vente 31

8.3.3 génération de facture : 32

8.4 Test de gestion des ordonnances 33

8.4.1 Vente avec ordonnance 33
8.4.2 Vente sans ordonnance 33
8.4.3 Vérification de l'enregistrement des informations du médecin et de la date 34
8.5 Test de gestion des commandes 34
8.5.1 Création d'une commande fournisseur 34
8.5.2 Ajout de plusieurs produits 35
8.5.3 Mise à jour du stock 35
8.6 Résultats obtenus 36

9 Limites et perspectives **37**

10 Conclusion **38**

Chapitre 1 : Introduction générale

Une pharmacie, au-delà de son rôle médical, est un lieu où s'entrecroisent des flux de données variés : entrées de stock, sorties par vente, suivi de péremption, prescriptions médicales, relations avec les fournisseurs. Quand ces informations sont gérées manuellement sur des registres papier ou des fichiers tableurs dispersés le risque d'erreur devient considérable. Un médicament périmé peut rester en rayon, une rupture de stock peut passer inaperçue, et la traçabilité des ordonnances se perd dans l'accumulation des documents.

Le sujet qui nous a été confié dans le cadre de ce module vise justement à répondre à cette problématique : concevoir et développer une application desktop capable de centraliser l'ensemble des opérations courantes d'une pharmacie. Le pharmacien doit pouvoir, depuis une seule interface, enregistrer ses médicaments, gérer ses clients, saisir des ordonnances, effectuer des ventes, passer des commandes auprès de ses fournisseurs et surveiller l'état de son stock.

Pour réaliser ce système, nous avons choisi Java comme langage de développement, Swing pour l'interface graphique, MySQL pour le stockage des données et JDBC pour la communication entre l'application et la base de données. Le projet suit une architecture en couches : Vue, Contrôleur, DAO, Modèle qui sépare clairement les responsabilités et facilite la maintenance du code.

Ce rapport retrace les étapes du projet : de l'analyse des besoins à la conception, puis de la réalisation technique aux tests, en passant par les difficultés rencontrées et les solutions adoptées. Nous terminerons par les limites de l'application et les perspectives d'amélioration envisagées.

Chapitre 2 : Analyse du besoin

2.1 Présentation du sujet

Le sujet demande la réalisation d'un système de gestion de pharmacie couvrant les opérations suivantes : gestion des médicaments et de leur stock, gestion des clients, saisie et vérification des ordonnances, enregistrement des ventes, suivi des commandes fournisseurs, historique des transactions et analyse des ventes.

L'objectif est de fournir un outil fonctionnel et intuitif permettant au pharmacien d'effectuer l'ensemble de ses tâches quotidiennes depuis une seule application, sans recourir à plusieurs outils distincts.

2.2 Besoins fonctionnels

Les besoins fonctionnels identifiés lors de l'analyse sont les suivants :

- Enregistrer et modifier les informations générales de la pharmacie.
- Ajouter, modifier, supprimer et rechercher des clients.
- Ajouter, modifier, supprimer et rechercher des médicaments.
- Suivre la quantité disponible en stock et détecter les seuils d'alerte.
- Détecter les médicaments périmés ou proches de la date de péremption.
- Gérer les fournisseurs (ajout, modification, suppression).
- Saisir les ordonnances et les associer à un client.
- Créer des ventes avec ou sans ordonnance.
- Mettre à jour automatiquement le stock après validation d'une vente.
- Créer des commandes fournisseurs et enregistrer leur réception.
- Augmenter le stock après réception d'une livraison.
- Générer automatiquement une facture texte après chaque vente validée.
- Consulter l'historique des ventes et des commandes.
- Analyser les ventes à travers un dashboard : par produit, par client, par période.

2.3 Besoins non fonctionnels

Critère	**Description**
Simplicité	L'interface doit être intuitive, exploitable sans formation préalable.
Fiabilité	Les données doivent rester cohérentes après chaque opération : stock, ventes, commandes.
Rapidité	Les opérations courantes doivent s'exécuter sans latence perceptible.
Persistance	Toutes les données sont stockées dans MySQL et conservées entre les sessions.

TABLE 2.1 – Besoins non fonctionnels

Chapitre 3 : Conception du système

3.1 Architecture du projet

Le projet adopte une architecture en quatre couches. Chaque couche a une responsabilité bien délimitée, ce qui facilite la maintenance et l'évolution du code.

Couche	Package	Responsabilité
Vue	`view`	Affichage : formulaires, tableaux, boutons et onglets.
Contrôleur	`controller`	Validation des données et application de la logique métier.
DAO	`dao`	Exécution des requêtes SQL et communication avec MySQL.
Modèle	`model`	Représentation des objets du domaine (Client, Médicament, etc.).

TABLE 3.1 – Couches de l'architecture

Le flux de données suit un chemin linéaire : l'utilisateur interagit avec la vue, la vue délègue au contrôleur, le contrôleur appelle le DAO, et le DAO communique avec MySQL. Les objets modèle circulent entre ces couches pour transporter les données.

Utilisateur $\longrightarrow$ **Vue (Swing)** $\longrightarrow$ **Contrôleur** $\longrightarrow$ **DAO (JDBC)** $\longrightarrow$ **MySQL**

FIGURE 3.1 – Flux de données dans l'architecture

3.2 Structure des packages

Le code source est organisé en cinq packages :

Package	Contenu
`Main`	Classe `Main` : point d'entrée de l'application.
`model`	Neuf classes : `Client`, `Medicament`, `Fournisseur`, `Pharmacie`, `Ordonnance`, `Vente`, `LigneVente`, `Commande`, `LigneCommande`.
`dao`	Un DAO par entité, plus `Utilitaire` (connexion) et `TestConnection`.
`controller`	Neuf contrôleurs dont `DashboardController` pour l'analyse des ventes.
`view`	Fenêtre principale `GestionPharmacieView` et un panel par module, dont `DashboardPanel`.

TABLE 3.2 – Organisation des packages

3.3 Classes principales du modèle

Le package `model` contient neuf classes qui représentent les entités du domaine :

Classe	Description
`Client`	Client avec nom, prénom, téléphone et adresse.
`Medicament`	Médicament avec nom commercial, composition, forme, dosage, prix, quantité en stock, date de péremption et seuil d'alerte.
`Fournisseur`	Fournisseur avec nom de société, contact et adresse.
`Pharmacie`	Informations de l'établissement : nom, adresse, téléphone, email, responsable.
`Ordonnance`	Ordonnance associée à un client, avec médecin, date et notes.
`Vente`	Vente avec client, ordonnance optionnelle, date et liste de lignes.
`LigneVente`	Détail d'un médicament vendu : identifiant vente, médicament, quantité, prix unitaire.
`Commande`	Commande fournisseur avec date, statut et liste de lignes.
`LigneCommande`	Détail d'un médicament commandé : identifiant commande, médicament, quantité, prix d'achat.

TABLE 3.3 – Classes du package `model`

3.4 Diagramme de cas d'utilisation

Le diagramme ci-dessous identifie les interactions entre les acteurs et le système. Le pharmacien accède aux fonctionnalités opérationnelles, tandis que le responsable se concentre sur l'analyse et le pilotage.

Pharmacien :
— Gérer les clients (ajout, modification, suppression)
— Gérer les médicaments
— Saisir une ordonnance
— Créer et valider une vente (avec ou sans ordonnance)
— Gérer le stock (entrées, sorties)
— Créer une commande fournisseur
— Réceptionner une livraison
— Générer une facture texte

Responsable :
— Consulter les historiques de ventes et de commandes
— Analyser le stock faible et la péremption
— Calculer la valeur totale du stock
— Consulter le dashboard : ventes par produit, par client, chiffre d'affaires par période
— Planifier les approvisionnements

FIGURE 3.2 – Cas d'utilisation du système

3.5 Diagramme de classes

Le diagramme de classes représente les neuf entités du modèle et leurs associations :

— Un `Client` peut posséder plusieurs `Ordonnance` ($1 \rightarrow 0..*$).
— Un `Client` peut effectuer plusieurs `Vente` ($1 \rightarrow 0..*$).
— Une `Ordonnance` peut justifier une `Vente` ($0..1 \rightarrow 0..*$).
— Une `Vente` contient une ou plusieurs `LigneVente` ($1 \rightarrow 1..*$).
— Une `LigneVente` concerne un `Medicament` ($0..* \rightarrow 1$).
— Un `Fournisseur` peut fournir plusieurs `Commande` ($1 \rightarrow 0..*$).
— Une `Commande` contient une ou plusieurs `LigneCommande` ($1 \rightarrow 1..*$).
— Une `LigneCommande` approvisionne un `Medicament` ($0..* \rightarrow 1$).

3.6 Diagramme entité-relation

Le diagramme entité-relation correspond directement à la structure de la base de données. Chaque entité devient une table, chaque association se traduit par une clé étrangère.

Chapitre 4 : Base de données

4.1 Présentation de la base

La base de données, nommée `gestion_pharmacie`, fonctionne sous MySQL. Elle contient neuf tables correspondant aux neuf entités du modèle. Le script de création définit les tables, les colonnes, les types de données et les contraintes d'intégrité.

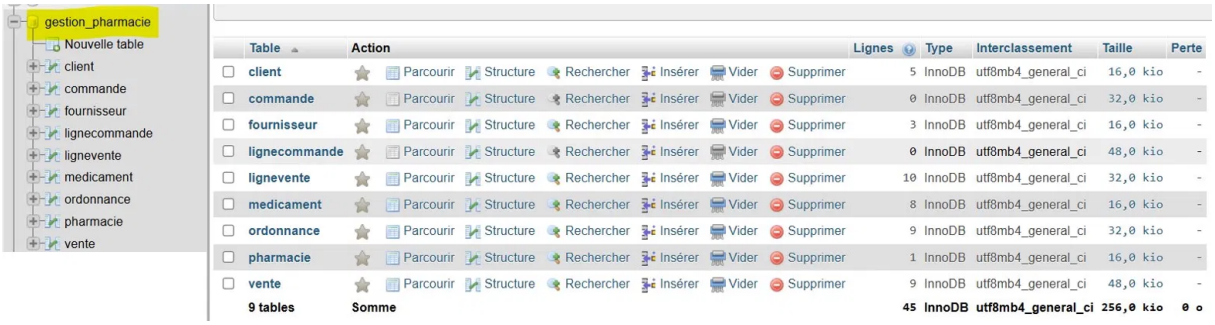


FIGURE 4.1 – Vue d'ensemble de la base de données`gestion_pharmacie` dans phpMyAdmin

4.2 Tables et relations

Table	Contenu et relations
`Client`	Clients de la pharmacie.
`Medicament`	Médicaments avec prix, stock et péremption.
`Fournisseur`	Fournisseurs de médicaments.
`Pharmacie`	Fiche de l'établissement.
`Ordonnance`	Ordonnances (`idClient` → `Client.id`).
`Vente`	Ventes (`idClient` → `Client.id`, `idOrdonnance` → `Ordonnance.id`).
`LigneVente`	Détail des ventes (`idVente` → `Vente.id`, `idMedicament` → `Medicament.id`).
`Commande`	Commandes fournisseurs (`idFournisseur` → `Fournisseur.id`).
`LigneCommande`	Détail des commandes (`idCommande` → `Commande.id`, `idMedicament` → `Medicament.id`).

TABLE 4.1 – Tables de la base de données

Les captures ci-après montrent la structure détaillée de chaque table telle qu'elle apparaît dans phpMyAdmin.



(a) Table `Client`



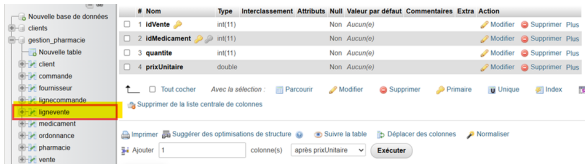
(b) Table `Commande`



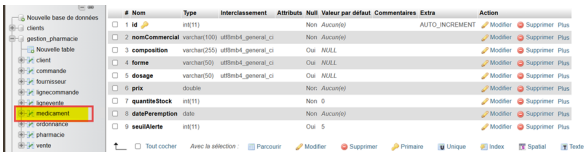
(a) Table `Fournisseur`



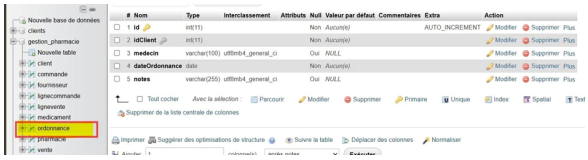
(b) Table `Lignecommande`



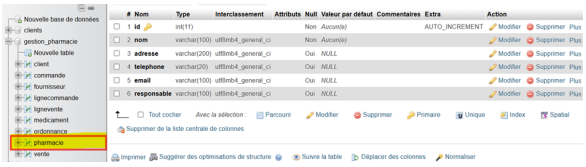
(a) Table `Lignevente`



(b) Table `Medicament`



(a) Table `Ordonnance`



(b) Table `Pharmacie`



FIGURE 4.6 – Table `Vente`

4.3 Clés primaires et étrangères

Chaque table possède une clé primaire `id` auto-incrémentée, sauf `LigneVente` et `LigneCommande` qui utilisent des clés composées (couple `idVente`/`idMedicament` et `idCommande`/`idMedicament`). Les clés étrangères assurent l'intégrité référentielle : il est impossible d'insérer une vente

référençant un client inexistant, ou une ligne de vente pointant vers un médicament supprimé.

Cette structuration garantit la cohérence des données à chaque opération. Par exemple, la suppression d'un client n'est possible que si aucune ordonnance ni aucune vente ne lui est rattachée.

4.4 Correspondance noms SQL et DAO

Au cours du développement, nous avons rencontré des incohérences entre les noms de colonnes définis dans les tables MySQL et les noms utilisés dans les requêtes SQL des classes DAO. Un attribut nommé `nomSociete` dans la table `Fournisseur` pouvait, par inattention, être écrit différemment dans le code Java. Ce décalage provoquait des exceptions `SQLException` empêchant l'affichage ou l'enregistrement des données.

La solution a consisté à comparer systématiquement la structure de chaque table avec les requêtes du DAO correspondant, puis à aligner les noms de colonnes. Ce travail minutieux a été réalisé sur l'ensemble des sept DAO du projet.

Chapitre 5 : Réalisation technique

5.1 Interface graphique : Java Swing

Pour construire l'interface de l'application, nous avons utilisé Java Swing, une bibliothèque graphique incluse dans Java. La fenêtre principale s'appelle `GestionPharmacieView`. Elle contient un composant `JTabbedPane`, qui fonctionne comme un classeur à onglets : chaque onglet correspond à un module de l'application. Il suffit de cliquer sur un onglet pour passer d'une fonction à une autre, sans ouvrir de nouvelle fenêtre.

Onglet	Fonction
Pharmacie	Formulaire pour enregistrer les informations de la pharmacie.
Clients	Ajouter, modifier, supprimer et rechercher des clients.
Médicaments	Ajouter, modifier, supprimer et rechercher des médicaments.
Fournisseurs	Gérer la liste des fournisseurs.
Ordonnances	Saisir et consulter les ordonnances.
Ventes	Créer une vente, gérer le panier et consulter l'historique.
Stock	Voir l'état du stock, les alertes de rupture et les dates de péremption.
Dashboard	Tableaux de statistiques : ventes par produit, par client et chiffre d'affaires.

TABLE 5.1 – Onglets de l'interface principale

La fenêtre s'ouvre avec une taille de 1000×650 pixels, se place automatiquement au centre de l'écran et se ferme complètement à la sortie grâce à `EXIT_ON_CLOSE`.

5.2 Accès aux données : JDBC

Pour que l'application puisse lire et écrire dans la base de données MySQL, on utilise JDBC (Java Database Connectivity), un pont entre Java et MySQL. Le fichier `mysql-connector-j-9.7.0.jar` assure cette connexion. Toute la gestion de la connexion est regroupée dans une seule classe appelée `Utilitaire`, ce qui évite de répéter le code de connexion dans chaque DAO.

Listing 5.1 – Classe Utilitaire — connexion MySQL

```
package dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Utilitaire {
    // Adresse de la base de donnees
    private static final String URL =
        "jdbc:mysql://localhost:3306/gestion_pharmacie";
    private static final String USER = "root";
    private static final String PASSWORD = "";
    private static Connection conn = null;

    // Ouvre la connexion si elle n'existe pas encore
    public static Connection getConnection() {
        try {
            if (conn == null || conn.isClosed()) {
                try {
                    Class.forName("com.mysql.cj.jdbc.Driver");
                    conn = DriverManager.getConnection(
                                URL, USER, PASSWORD);
                } catch (ClassNotFoundException e) {
                    System.out.println("Driver non trouve : "
                                        + e.getMessage());
                }
            }
        } catch (SQLException e) {
            System.out.println("Erreur connexion : "
                                + e.getMessage());
        }
        return conn;
    }

    // Ferme la connexion proprement
    public static void closeConnection() {
        try {
            if (conn != null && !conn.isClosed()) conn.close();
        } catch (SQLException e) {
            System.err.println("Erreur fermeture : "
```

```
41                                      + e.getMessage());
42              }
43          }
44  }
```

Cette classe ouvre une seule connexion partagée par tous les DAO (principe du *singleton*). La méthode `getConnection()` vérifie d'abord si la connexion existe déjà avant d'en créer une nouvelle. La méthode `closeConnection()` libère la connexion quand l'application n'en a plus besoin.

5.3 Couche DAO

Chaque entité du projet possède son propre DAO (Data Access Object). Un DAO est simplement une classe qui sait comment lire et écrire les données d'une entité dans la base de données. On utilise des `PreparedStatement` à la place de simples chaînes de texte SQL, ce qui protège contre les attaques par injection SQL.

Toutes les classes DAO partagent les cinq opérations de base :

- `ajouter()` : enregistre un nouvel élément dans la base.
- `modifier()` : met à jour un élément existant.
- `supprimer()` : efface un élément de la base.
- `getAll()` : retourne la liste complète des éléments.
- `getById()` : retrouve un seul élément par son identifiant.

Certains DAO ont des méthodes en plus selon leurs besoins. Par exemple, `VenteDao` sait enregistrer les lignes d'une vente (`enregistrerLigne`), mettre à jour le stock après une vente, et rattacher une ordonnance à une vente existante (`modifierOrdo`). De son côté, `CommandeDao` gère le changement de statut d'une commande et la mise à jour du stock lors de la réception d'une livraison.

5.4 Couche Contrôleur

Les contrôleurs jouent le rôle de chef d'orchestre : ils reçoivent les actions de l'utilisateur depuis la vue, vérifient que tout est correct, puis demandent au DAO d'agir sur la base de données. Cette organisation évite de mélanger les règles de gestion avec l'affichage ou les requêtes SQL.

Voici ce que fait chaque contrôleur principal :

- `VenteController` : vérifie que le stock est suffisant avant de valider une vente, s'assure que l'ordonnance appartient bien au bon client, et crée automatiquement

la facture après validation.

— `StockController` : calcule la valeur totale du stock et repère les médicaments en rupture ou sur le point d'expirer.

— `CommandeController` : fait avancer le statut d'une commande (En attente → Validée → Livrée) et augmente le stock à la réception.

— `DashboardController` : regroupe les données de ventes pour produire des chiffres par produit, par client et par période.

5.5 Génération des factures texte

Dès qu'une vente est validée, le programme crée automatiquement un fichier texte qui sert de facture. C'est la méthode `genererEtEnregistrerFacture()` du `VenteController` qui s'en charge. Elle assemble le contenu de la facture (numéro de vente, client, date, liste des articles achetés, total) et l'enregistre dans un fichier nommé `facture_<id>.txt`, placé dans un dossier `factures/` créé automatiquement s'il n'existe pas encore.

Chaque facture contient :

— Le numéro de la vente.

— L'identifiant du client.

— La date de vente.

— La mention de l'ordonnance (son numéro, ou « Sans ordonnance » si absente).

— Le détail de chaque ligne : nom du médicament, quantité, prix unitaire et sous-total.

— Le total à payer en dirhams.

5.6 Point d'entrée

Le démarrage de l'application se fait depuis la classe `Main`. Elle utilise `SwingUtilities.invokeLater()` pour lancer l'interface graphique dans le bon fil d'exécution, comme l'exige Swing.

Listing 5.2 – Classe Main — démarrage de l'application

```
public class Main {
    public static void main(String[] args) {
        // Lance l'interface dans le bon fil d'execution Swing
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                GestionPharmacieView fenetre =
                    new GestionPharmacieView();
                fenetre.setVisible(true);
            }
```

```
            });
        }
}
```

Chapitre 6 : Fonctionnement de l'application

6.1 Gestion des médicaments

Ajout d'un médicament : L'utilisateur ouvre l'onglet « Médicaments », remplit le formulaire avec le nom commercial, la composition, la forme, le dosage, le prix, la quantité en stock, la date de péremption et le seuil d'alerte, puis clique sur « Ajouter ». Le MedicamentController valide les champs et appelle le DAO qui exécute un INSERT dans la table Medicament. Le tableau se rafraîchit automatiquement pour afficher le nouveau produit.

Modification d'un médicament : L'utilisateur clique sur une ligne du tableau pour pré-remplir le formulaire, modifie les champs souhaités, puis clique sur « Modifier ». Le MedicamentController appelle le DAO qui exécute un UPDATE sur l'enregistrement correspondant. Le tableau se rafraîchit pour refléter les nouvelles valeurs.

Suppression d'un médicament : L'utilisateur sélectionne un médicament dans le tableau puis clique sur « Supprimer ». Le MedicamentController appelle le DAO qui exécute un DELETE dans la table Medicament. Le formulaire est vidé et le tableau se met à jour.

Recherche d'un médicament : L'utilisateur saisit un mot-clé dans la barre de recherche et clique sur « Rechercher ». Le MedicamentController parcourt la liste des médicaments et retourne ceux dont le nom commercial contient le mot-clé. Les résultats s'affichent dans le tableau.

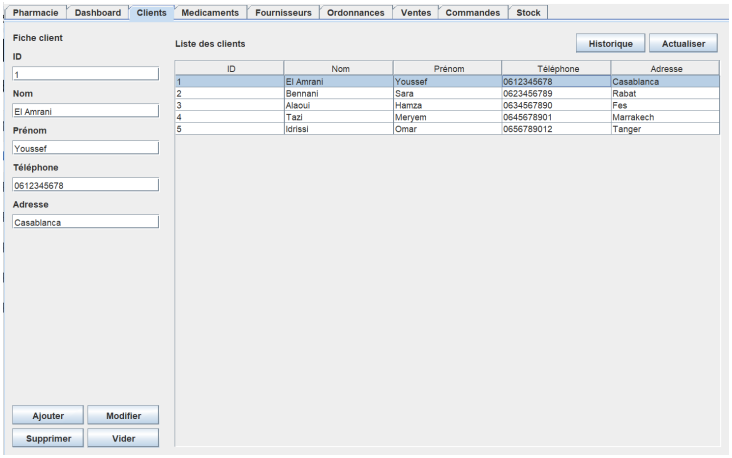


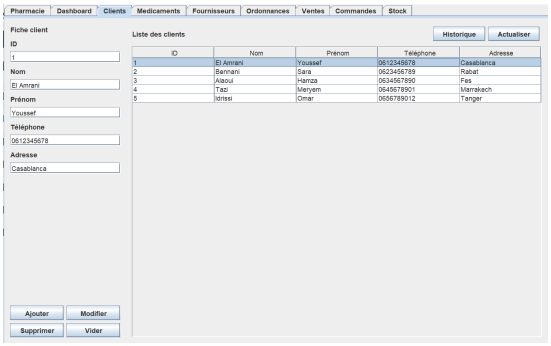
FIGURE 6.1 – L'onglet Médicaments

6.2 Gestion des clients

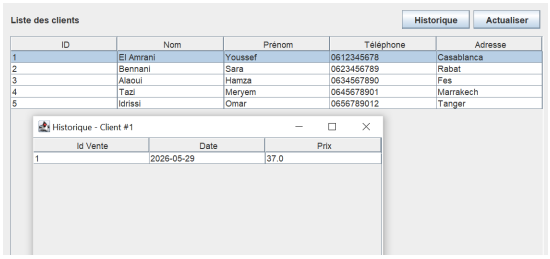
Ajout d'un client : L'utilisateur ouvre l'onglet « Clients », remplit le formulaire avec le nom, le prénom, le téléphone et l'adresse, puis clique sur « Ajouter ». Le ClientController appelle le DAO qui exécute un INSERT dans la table Client. Le tableau se rafraîchit pour afficher le nouveau client.

Modification et suppression : L'utilisateur sélectionne un client dans le tableau pour pré-remplir le formulaire, effectue ses modifications et clique sur « Modifier » ou « Supprimer ». Le ClientController appelle le DAO qui exécute respectivement un UPDATE ou un DELETE dans la table Client.

Historique des achats : L'utilisateur sélectionne un client puis clique sur « Historique ». Le ClientController appelle listerHistoriqueAchats() qui récupère toutes les ventes associées à ce client. Une nouvelle fenêtre s'ouvre et affiche l'identifiant de chaque vente, sa date et son montant total.



(a) L'onglet Clients



(b) Historique d'un client

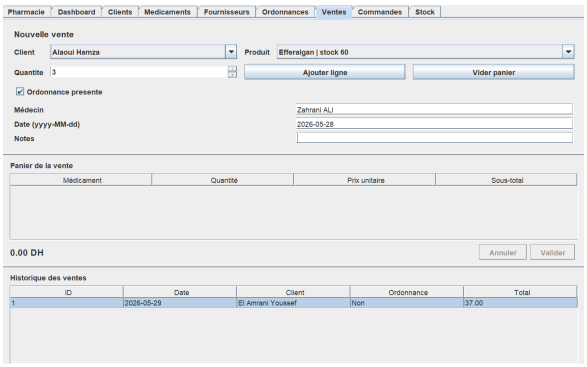
6.3 Gestion des ventes

Création d'une vente : L'utilisateur ouvre l'onglet « Ventes », sélectionne un client dans la liste déroulante et choisit un médicament avec sa quantité. Dès le premier clic sur « Ajouter ligne », le VenteController appelle creerVente() qui exécute un INSERT dans la table Vente avec le statut en cours. Chaque ligne ajoutée est enregistrée dans la table LigneVente via creerLigneVente(). Le panier se met à jour visuellement avec le sous-total de chaque ligne et le total général.

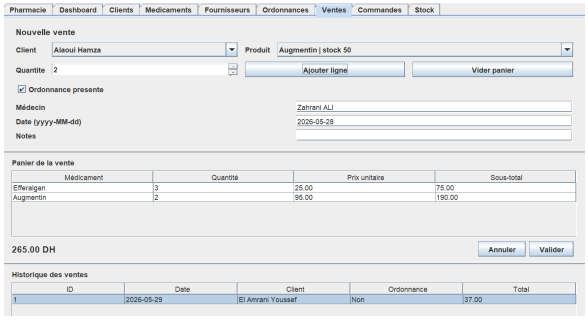
Ajout d'une ordonnance : Si la vente nécessite une ordonnance, l'utilisateur coche « Ordonnance présente » et remplit le nom du médecin, la date et les notes. Au moment de la validation, le VentePanel appelle enregistrerOrdonnanceSiPresente() qui crée l'ordonnance via OrdonnanceController.ajouterOrdonnance() et associe son identifiant à la vente avant de la valider.

Validation d'une vente : L'utilisateur clique sur « Valider ». Le VenteController appelle enregistrerVente() qui vérifie que l'ordonnance appartient bien au client sélectionné et que le stock est suffisant pour chaque médicament du panier. Si les vérifications réussissent, le DAO exécute un UPDATE sur la table Medicament pour décrémenter le stock de chaque ligne. Le panier est vidé et l'historique se rafraîchit.

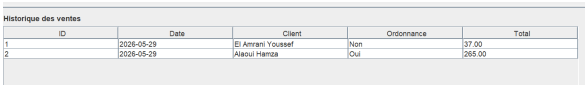
Annulation d'une vente : L'utilisateur clique sur « Annuler ». Le VenteController appelle annulerVente() qui exécute un DELETE dans les tables Vente et LigneVente. Aucune modification de stock n'est effectuée puisque la vente n'avait pas encore été validée.



(a) L'onglet Vente



(b) Lignes de la Vente



(c) Historique des ventes

6.4 Génération automatique de facture

Immédiatement après la validation de la vente, la méthode `genererEtEnregistrerFacture()` est appelée. Elle produit un fichier `facture_<id>.txt` contenant l'en-tête, le détail des achats et le total. Ce fichier est stocké sur le disque dans le dossier `factures/`, créé automatiquement si nécessaire.

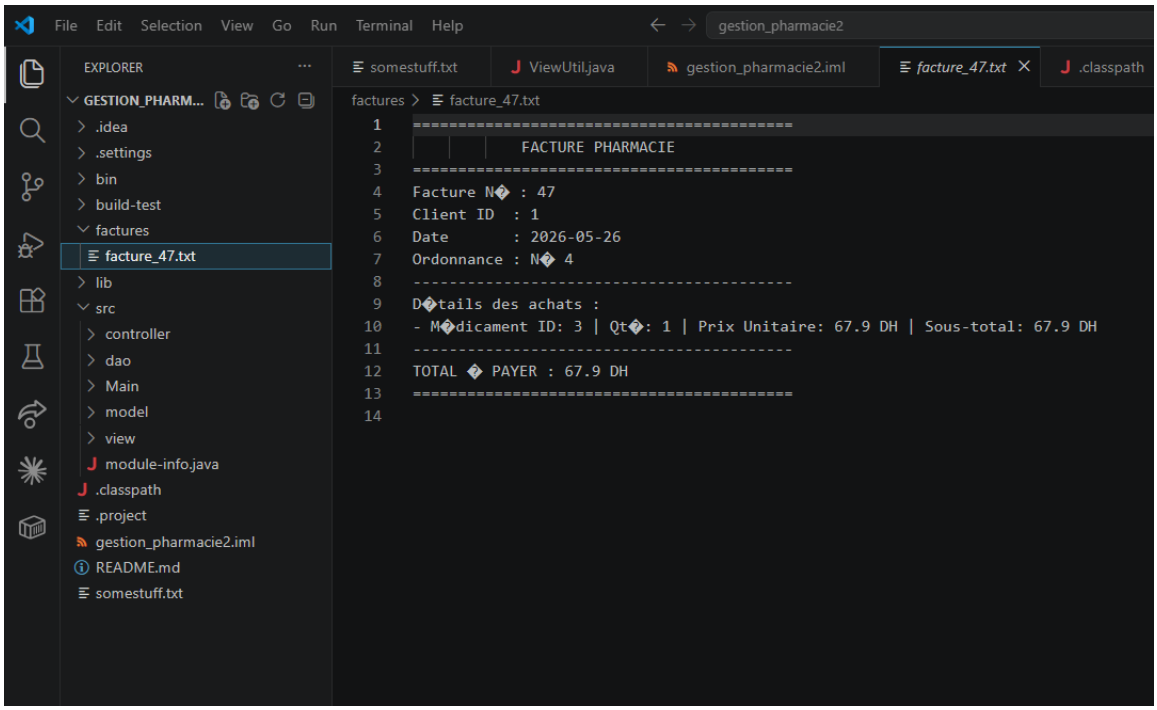


FIGURE 6.4 – Exemple de facture générée automatiquement après une vente

6.5 Commande fournisseur et réception

Le pharmacien crée une commande en sélectionnant un fournisseur, un médicament, une quantité et un prix d'achat. La commande est enregistrée avec le statut « En attente ». Il peut ensuite la valider (statut « Validée »), puis la réceptionner (statut « Livrée »). Lors de la réception, le stock de chaque médicament commandé est automatiquement augmenté de la quantité livrée.

Le `CommandePanel` affiche également un historique des commandes avec le fournisseur, la date, le statut et le total.

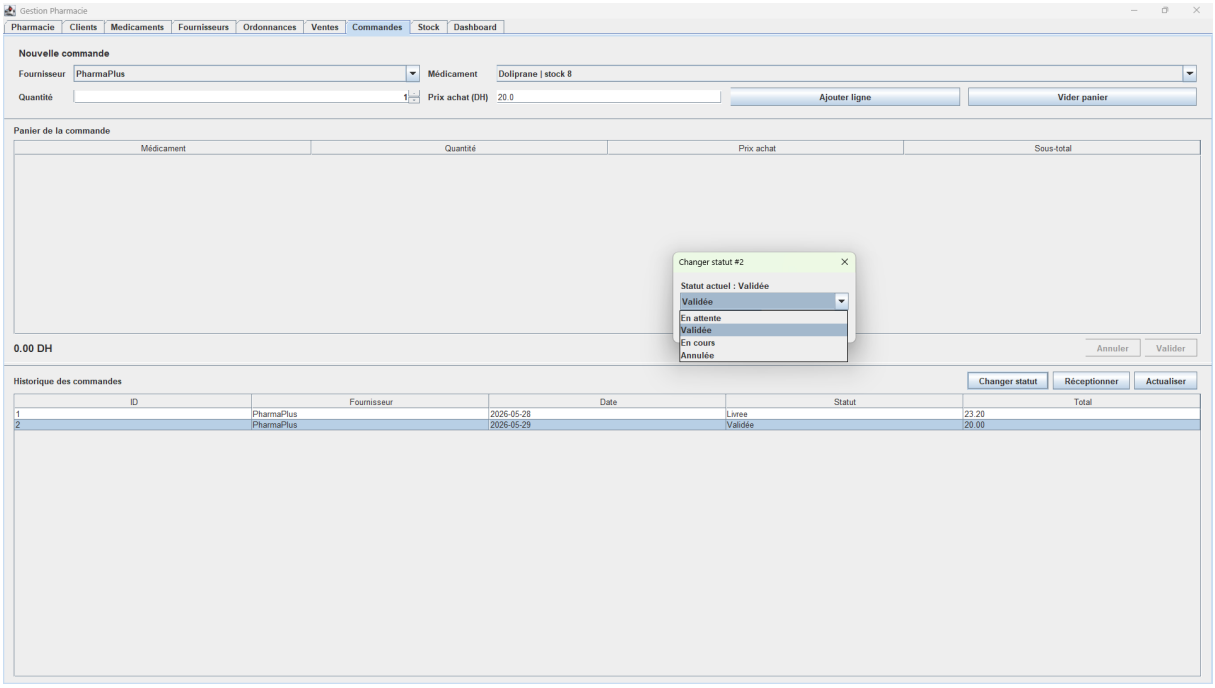


FIGURE 6.5 – Interface de gestion des commandes fournisseurs et changement de statut

6.6 Suivi du stock et alertes

L'onglet « Stock » offre une vue d'ensemble sur les médicaments disponibles. Le `StockController` propose les fonctions suivantes :

- **Stock faible** : affiche les médicaments dont la quantité est inférieure ou égale au seuil d'alerte.
- **Produits périmés** : affiche les médicaments dont la date de péremption est dépassée.
- **Péremption proche** : affiche les médicaments qui expireront dans les 30 prochains jours.
- **Valeur du stock** : calcule la somme $prix \times quantité$ pour l'ensemble des médicaments.

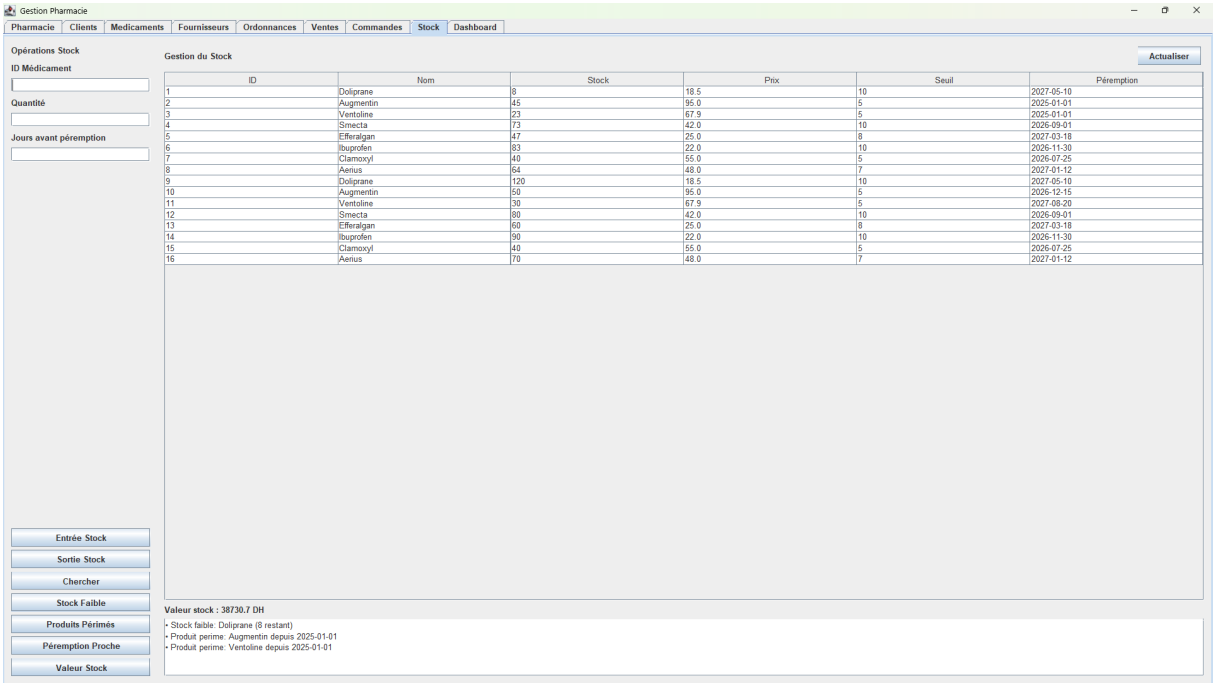


FIGURE 6.6 – Interface de suivi du stock, des alertes et de la valeur totale

6.7 Analyse avec le dashboard

Le module Dashboard, accessible depuis l'onglet éponyme, centralise plusieurs indicateurs construits à partir des ventes enregistrées :

- **Ventes par produit** : un tableau affiche, pour chaque médicament, la quantité totale vendue.
- **Ventes par client** : un tableau affiche, pour chaque client, le montant total de ses achats en dirhams.
- **Produits les plus demandés** : les médicaments sont classés par quantité vendue décroissante.
- **Chiffre d'affaires par période** : l'utilisateur saisit une date de début et une date de fin au format `yyyy-mm-dd`, puis clique sur « Calculer CA ». Le résultat s'affiche dans un champ dédié.
- Le bouton « Actualiser tout » recharge l'ensemble des statistiques depuis la base.

Le `DashboardController` exploite les données de `VenteController`, `ClientController` et `MedicamentController` pour agréger les résultats. Le `DashboardPanel` utilise des sous-onglets `JTabbedPane` pour organiser les différentes vues d'analyse.

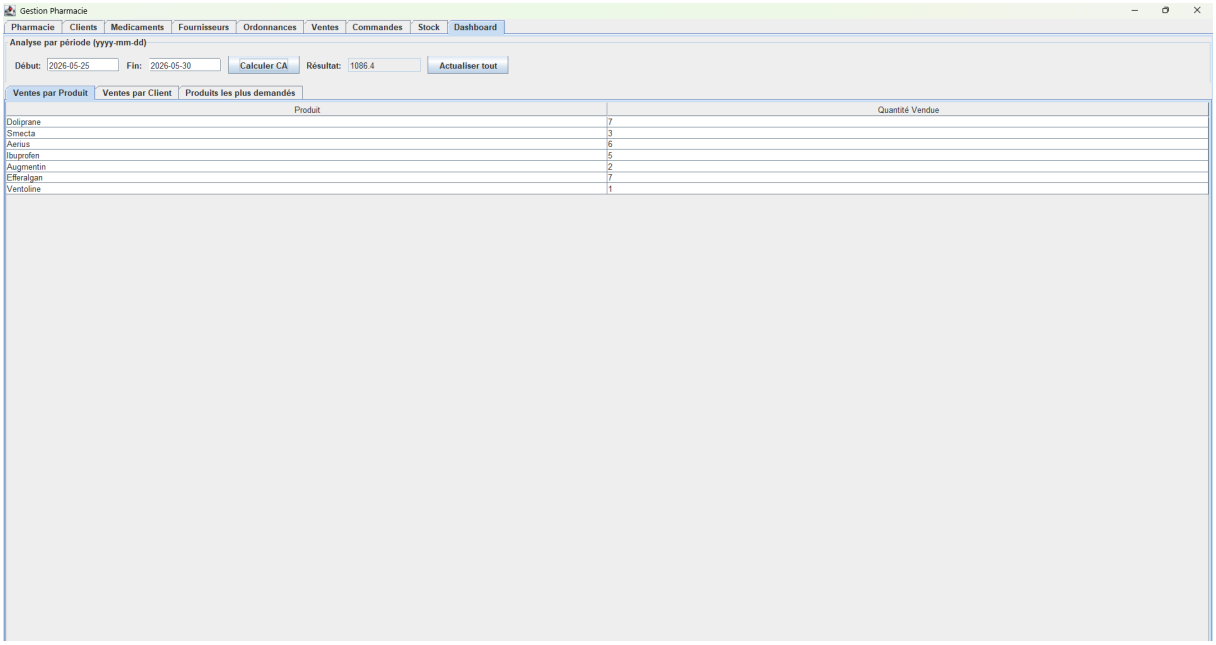


FIGURE 6.7 – Dashboard d'analyse des ventes par produit et chiffre d'affaires par période

Chapitre 7 : Problèmes rencontrés

Au cours du développement de l'application de gestion de pharmacie, plusieurs difficultés techniques et fonctionnelles ont été rencontrées. Chaque difficulté rencontrée nous a permis d'améliorer progressivement l'application.

7.1 Modélisation des ventes et des lignes de vente

Cause : Au début du développement, il n'était pas clair comment représenter une vente contenant plusieurs médicaments, avec pour chacun une quantité et un prix unitaire. La structure de données initiale ne permettait pas de gérer correctement cette relation entre la vente globale et ses éléments détaillés.

Impact : Sans une modélisation adaptée, il devenait difficile d'enregistrer plusieurs produits dans une même vente, de calculer le montant total et d'assurer une bonne cohérence des données. Le même problème se posait également pour les commandes.

Solution : Nous avons finalement adopté une structure plus adaptée en créant une classe LigneVente séparée, reliée à la classe Vente. Chaque ligne contient les informations relatives à un médicament, notamment sa quantité et son prix unitaire. Une approche similaire a été appliquée pour les commandes avec la classe LigneCommande, ce qui a permis de mieux organiser les données et de faciliter la gestion des ventes et des commandes.

7.2 Intégration de l'ordonnance dans la vente

Cause : Au début, il fallait trouver comment relier une ordonnance à la fois au client et à la vente. Nous avons aussi dû faire en sorte que les informations de l'ordonnance ne soient saisies qu'au bon moment.

Impact : Il y avait un risque d'enregistrer une ordonnance avant même que la vente soit validée, ce qui pouvait créer des incohérences dans les données.

Solution : Nous avons ajouté une option pour signaler qu'une ordonnance est présente dans la fenêtre de vente. Quand cette option est activée, les champs nécessaires apparaissent. L'ordonnance est ensuite enregistrée seulement au moment où la vente est confirmée.

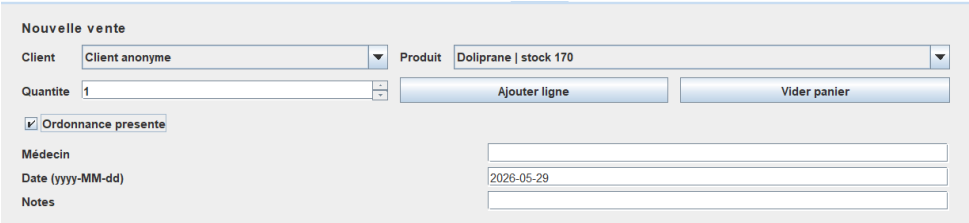


FIGURE 7.1 – Ordonnance

7.3 Ajout d'une ligne avant création de la vente

Cause : Une ligne de vente nécessite un `idVente` pour être insérée dans la table `LigneVente`. Or, la vente n'existait pas encore lorsque l'utilisateur ajoutait la première ligne.

Impact : L'insertion échouait car la clé étrangère `idVente` était invalide.

Solution : Au premier ajout de ligne, la vue crée automatiquement une vente dans la base et conserve son identifiant dans `idVenteEnCours`. Les lignes suivantes réutilisent cet identifiant.

7.4 Association tardive de l'ordonnance à la vente

Cause : L'ordonnance est créée au moment de la validation, alors que la vente existe déjà depuis le premier ajout de ligne. Il fallait donc pouvoir modifier la vente après sa création initiale.

Impact : La vente restait sans ordonnance même lorsque le pharmacien en avait saisi une.

Solution : Ajout de la méthode `modifierOrdo()` dans `VenteController` et `VenteDao` pour mettre à jour le champ `idOrdonnance` dans la table `Vente` après l'enregistrement de l'ordonnance.

7.5 Génération et stockage des factures texte

Cause : La facture devait être générée sous forme de fichier texte et conservée physiquement sur le disque après validation de la vente.

Impact : Sans cette fonctionnalité, aucune trace imprimable de la transaction n'était conservée.

abdSolution : Ajout de la méthode `genererEtEnregistrerFacture()` dans `VenteController`. Elle crée le dossier `factures/` si nécessaire et y écrit un fichier `facture_<id>.txt` contenant le détail complet de la vente.

7.6 Séparation des fichiers DAO et Controller

Cause : Au début du projet, nous ne savions pas comment organiser les différentes parties du code. Les traitements de l'application et les accès à la base de données étaient parfois regroupés, ce qui rendait le code moins clair.

Impact : Le projet devenait plus difficile à comprendre et à modifier. Lorsqu'une erreur apparaissait, il était aussi plus compliqué de trouver son origine.

Solution : Nous avons décidé de séparer le projet en deux parties principales. Les classes DAO s'occupent uniquement de la communication avec la base de données, tandis que les classes Controller gèrent les traitements et les actions de l'utilisateur. Cette organisation a rendu le code plus clair, plus facile à maintenir et à développer par la suite.

Chapitre 8 : Tests et validation

Après le développement des différentes fonctionnalités, plusieurs tests ont été réalisés afin de vérifier le bon fonctionnement de l'application. Ces tests ont permis de s'assurer que les opérations principales sont exécutées correctement et que les données sont enregistrées de manière cohérente dans la base de données.

8.1 Test de gestion des clients

8.1.1 Ajout d'un client

Nous avons commencé par tester l'ajout d'un nouveau client en remplissant tous les champs requis (nom, prénom, numéro de téléphone, etc.). Après validation du formulaire, le client apparaît immédiatement dans la liste et ses informations sont bien enregistrées dans la base de données.

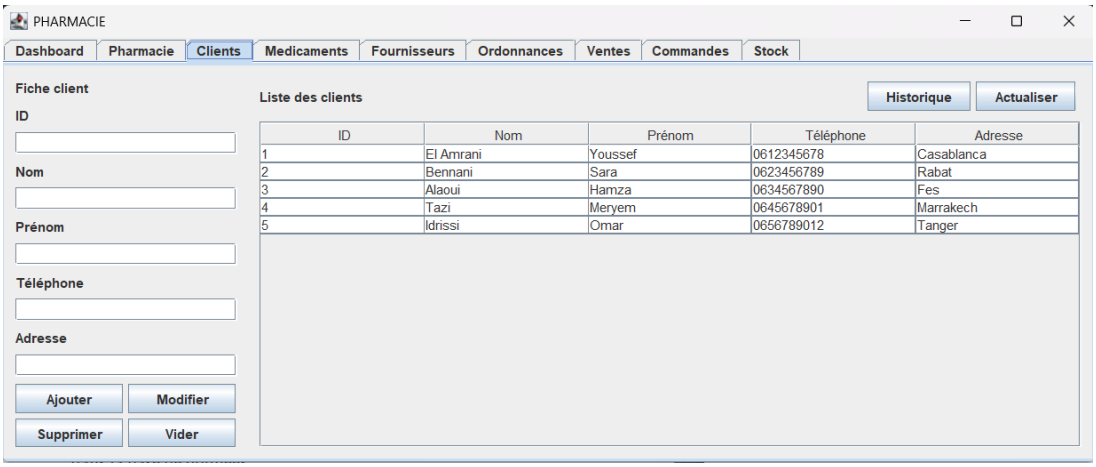


FIGURE 8.1 – Ajout d'un nouveau client dans le système

8.1.2 Modification des informations

Nous avons ensuite sélectionné un client existant et modifié certaines de ses informations, comme le numéro de téléphone ou l'adresse. Après avoir confirmé la modification, les nouvelles données sont bien mises à jour sans affecter les autres enregistrements.

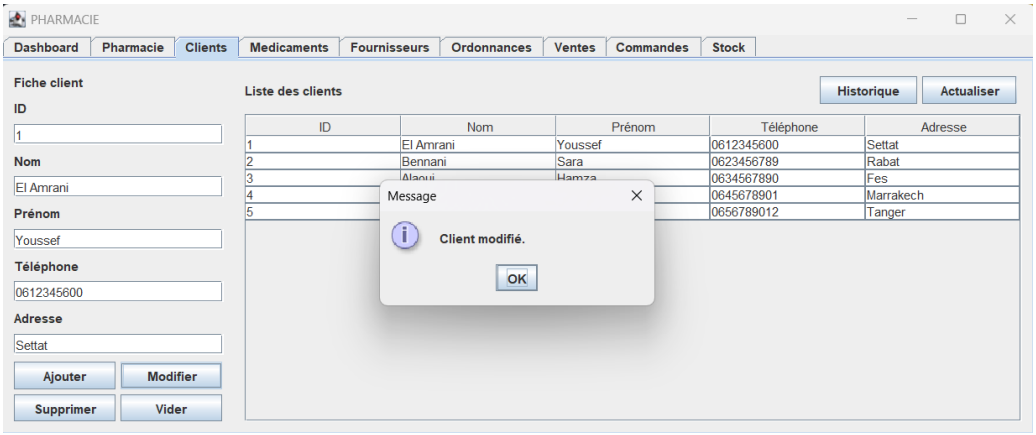


FIGURE 8.2 – Modification les informations du client Youssef

8.1.3 Suppression d'un client

Pour tester la suppression, nous avons choisi un client et cliqué sur le bouton de suppression. Un message de confirmation s'affiche avant de procéder, ce qui évite toute suppression accidentelle. Une fois confirmée, le client est retiré de la liste et de la base de données.

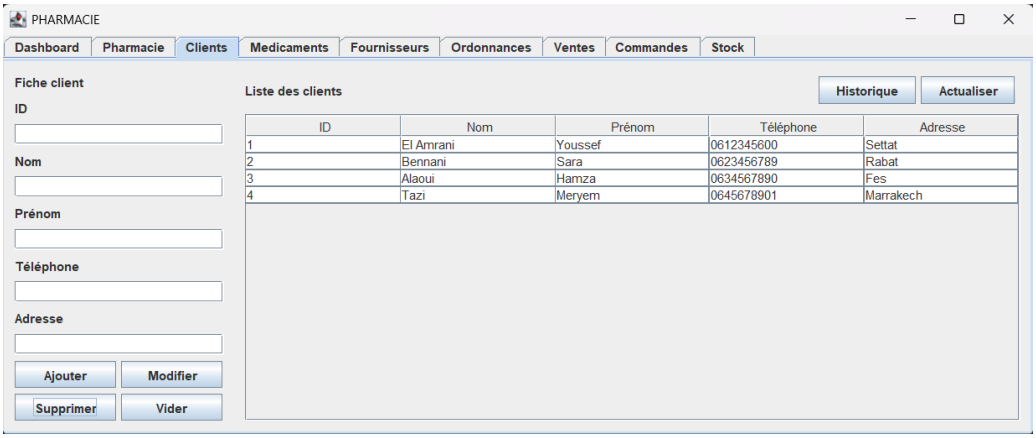


FIGURE 8.3 – Suppression du cient "Omar"

8.2 Test de gestion des médicaments

8.2.1 Ajout d'un médicament

Nous avons ajouté un nouveau médicament en renseignant les informations nécessaires : nom du produit, catégorie, prix unitaire, quantité en stock et date d'expiration. Le médicament s'affiche correctement dans la liste après la validation du formulaire.

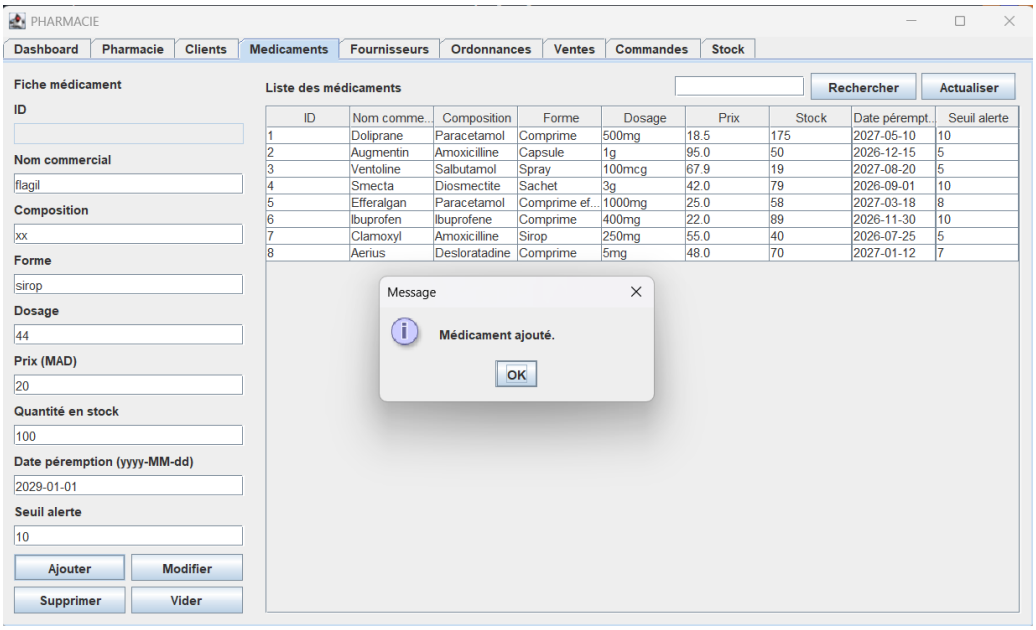


FIGURE 8.4 – Ajout d'un nouveau médicament

8.2.2 Recherche par nom

Nous avons vérifié qu'il est possible de modifier la quantité en stock d'un médicament déjà existant. Après la mise à jour, la nouvelle quantité est immédiatement visible dans la fiche du produit et dans la base de données.

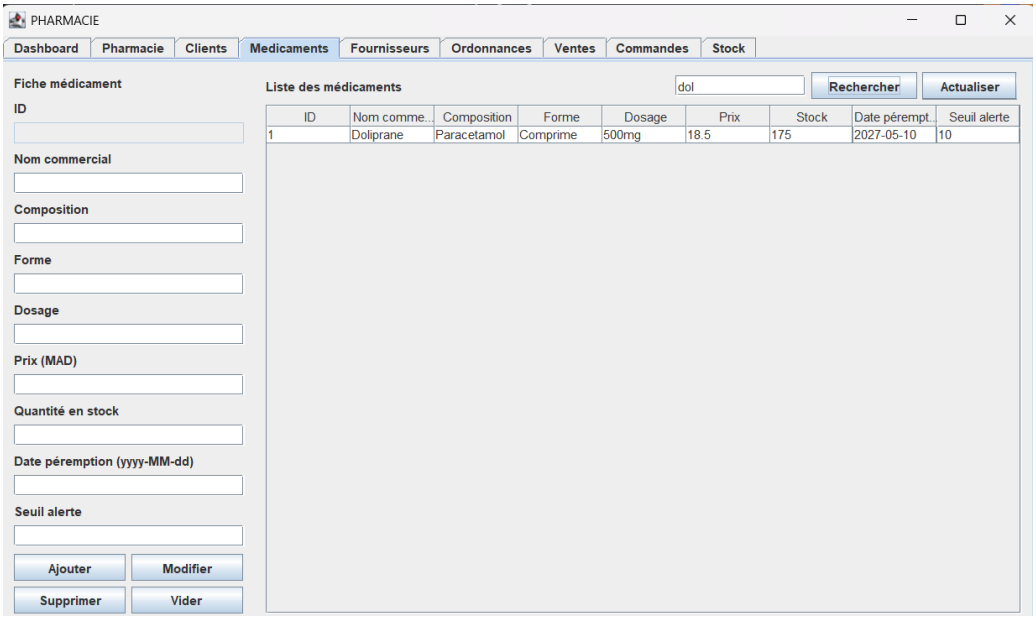


FIGURE 8.5 – Recherche par nom d'un médicament

8.3 Test de gestion des ventes

8.3.1 Création d'une vente

Nous avons simulé une vente en sélectionnant un client, puis en ajoutant un médicament au panier. Le processus est fluide et la vente est enregistrée dans la base de données après la validation.

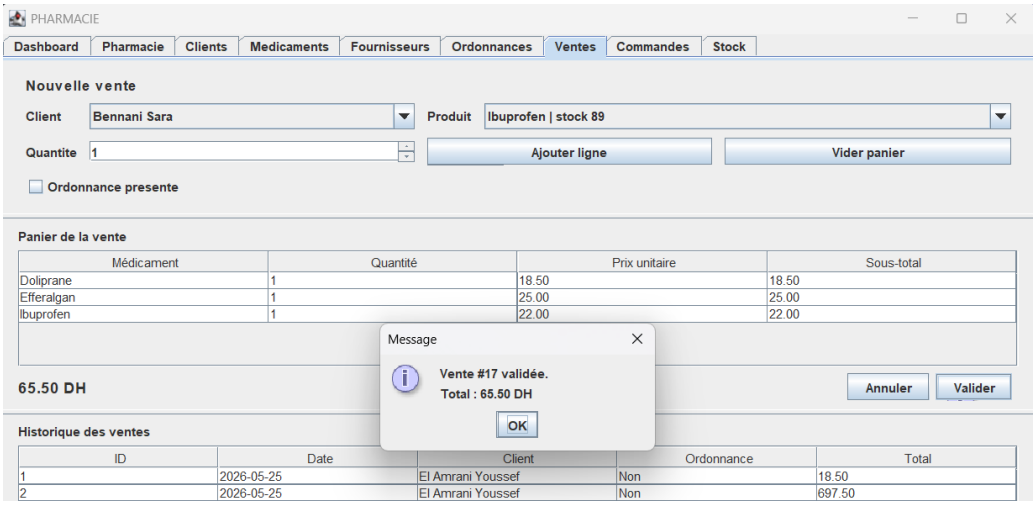


FIGURE 8.6 – Création d'une nouvelle vente

8.3.2 Ajout de plusieurs médicaments dans une même vente

Nous avons testé l'ajout de plusieurs lignes de vente dans une seule transaction. Chaque médicament ajouté apparaît sur une ligne distincte avec sa quantité et son prix. Ce fonctionnement correspond bien au modèle de données que nous avons mis en place avec la classe LigneVente.

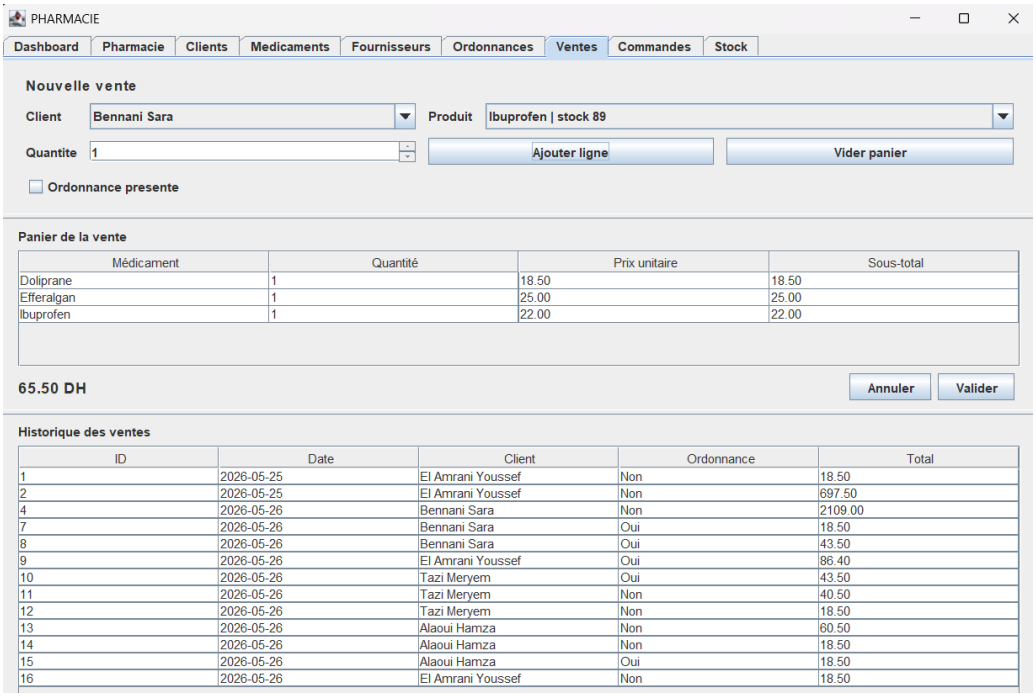


FIGURE 8.7 – Vente contenant plusieurs lignes de médicaments

8.3.3 génération de facture :

Après avoir validé une vente, nous avons vérifié que la facture a été bien générer dans le dossier factures ;

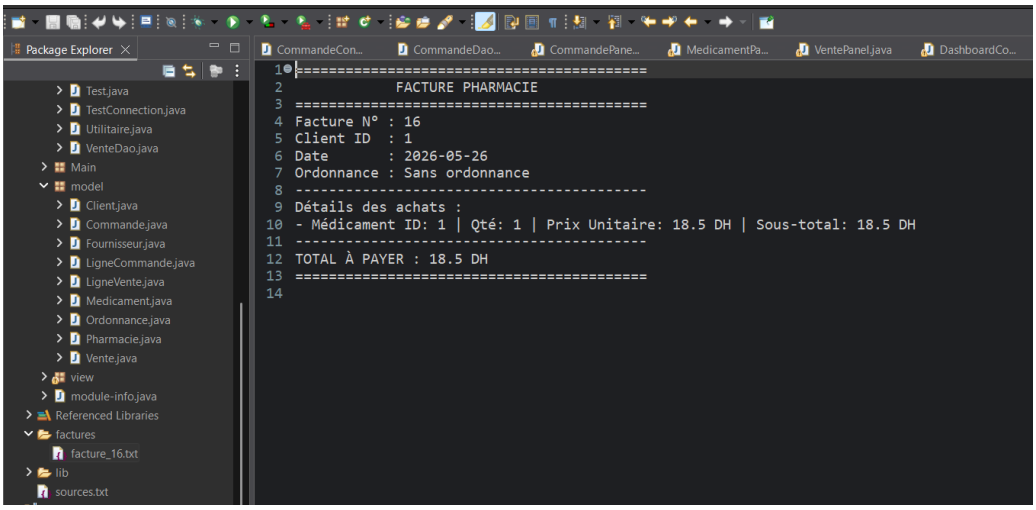


FIGURE 8.8 – facture généré automatiquement après validation de la vente

8.4 Test de gestion des ordonnances

8.4.1 Vente avec ordonnance

Nous avons testé le cas où le client présente une ordonnance. En activant l'option correspondante dans la fenêtre de vente, les champs relatifs à l'ordonnance apparaissent (nom du médecin, date de prescription). Après validation, l'ordonnance est correctement enregistrée et rattachée à la vente concernée.

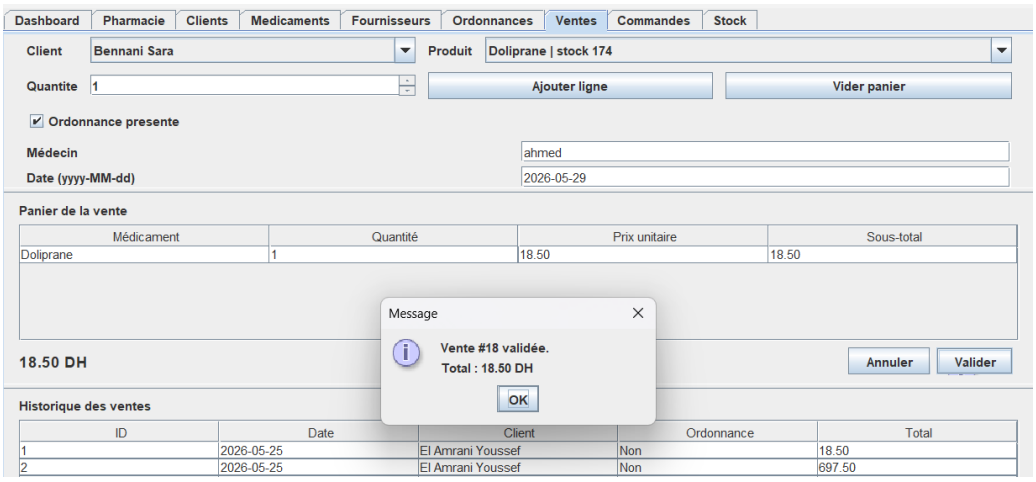


FIGURE 8.9 – Enregistrement d'une vente avec ordonnance

8.4.2 Vente sans ordonnance

Dans le cas d'une vente libre, sans ordonnance, le formulaire reste simplifié. Aucun champ lié à l'ordonnance n'est affiché, et la vente se déroule normalement sans blocage.

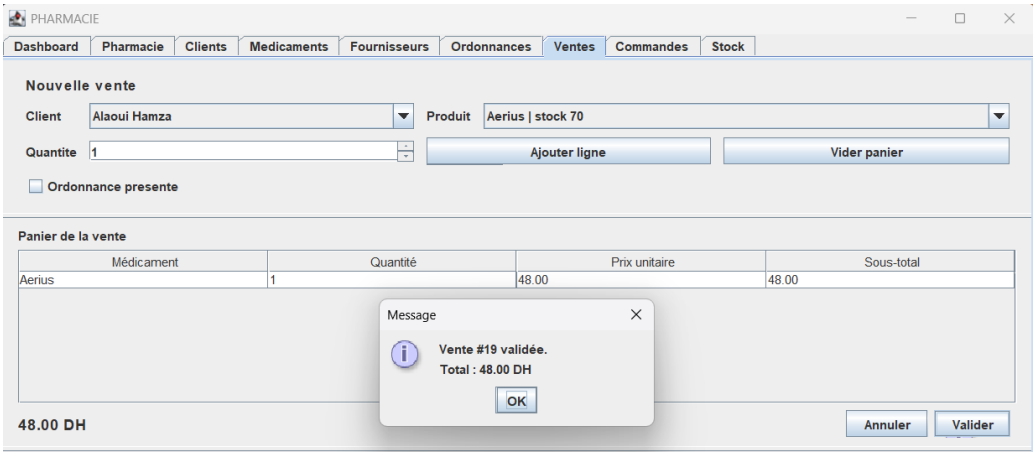
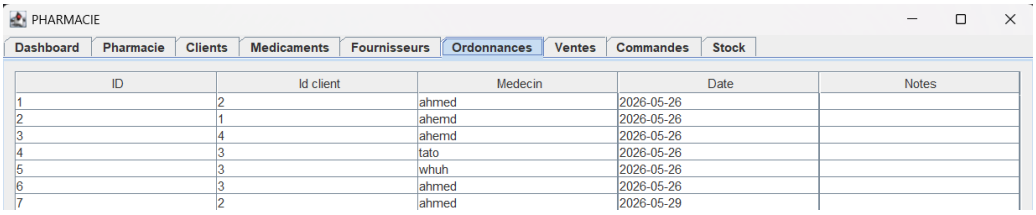


FIGURE 8.10 – Vente effectuée sans ordonnance

8.4.3 Vérification de l'enregistrement des informations du médecin et de la date

Nous avons vérifié dans la base de données que les informations saisies lors d'une vente avec ordonnance (nom du médecin prescripteur et date de prescription) sont bien sauvegardées et consultables ultérieurement. Cette traçabilité est essentielle dans le contexte pharmaceutique.



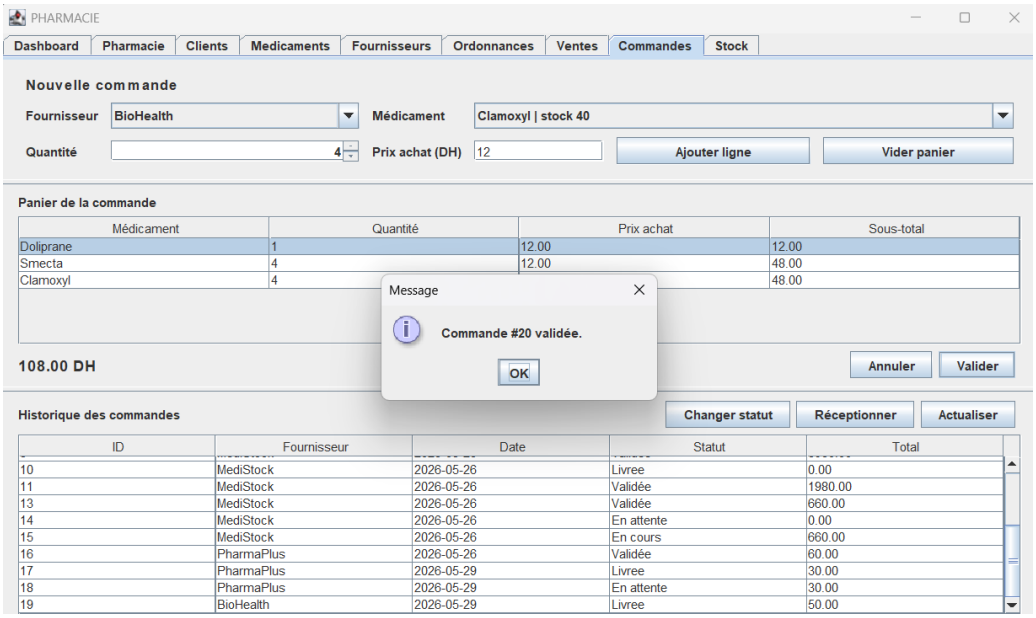
ID	Id client	Medecin	Date	Notes
1	2	ahmed	2026-05-26	
2	1	ahemd	2026-05-26	
3	4	ahemd	2026-05-26	
4	3	tato	2026-05-26	
5	3	whuh	2026-05-26	
6	3	ahmed	2026-05-26	
7	2	ahmed	2026-05-29	

FIGURE 8.11 – Enregistrement d'une vente avec ordonnance

8.5 Test de gestion des commandes

8.5.1 Création d'une commande fournisseur

Nous avons créé une commande en sélectionnant un fournisseur parmi ceux enregistrés dans le système. La commande reçoit un identifiant unique et une date de création, ce qui permet de la retrouver facilement par la suite.



ID	Fournisseur	Date	Statut	Total
10	MediStock	2026-05-26	Livree	0.00
11	MediStock	2026-05-26	Validée	1980.00
13	MediStock	2026-05-26	Validée	660.00
14	MediStock	2026-05-26	En attente	0.00
15	MediStock	2026-05-26	En cours	660.00
16	PharmaPlus	2026-05-26	Validée	60.00
17	PharmaPlus	2026-05-29	Livree	30.00
18	PharmaPlus	2026-05-29	En attente	30.00
19	BioHealth	2026-05-29	Livree	50.00

FIGURE 8.12 – Création d'une commande fournisseur

8.5.2 Ajout de plusieurs produits

Comme pour les ventes, il est possible d'ajouter plusieurs lignes de produits dans une même commande grâce à la classe LigneCommande. Chaque produit est associé à une quantité demandée et le tout est affiché clairement dans l'interface.

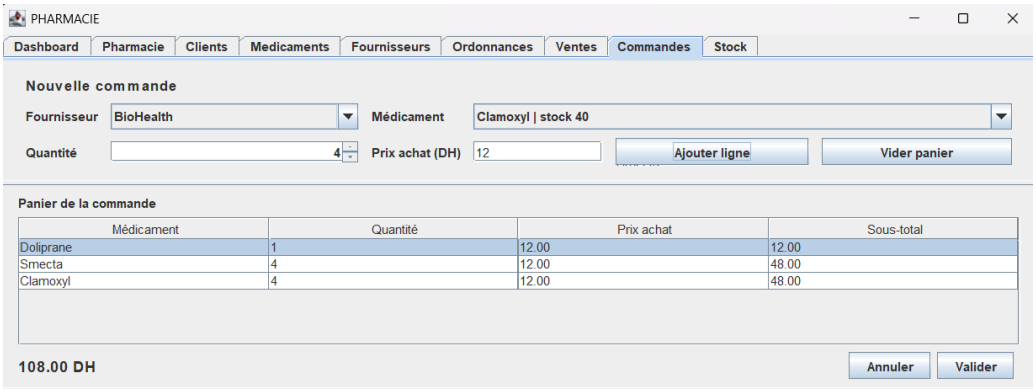


FIGURE 8.13 – Commande contenant plusieurs lignes de produits

8.5.3 Mise à jour du stock

Après la réception de la commande, les quantités des produits commandés sont automatiquement ajoutées au stock existant. Ce mécanisme est l'inverse de celui des ventes : au lieu de diminuer le stock, la commande vient le réapprovisionner, garantissant ainsi une gestion complète du cycle de vie des produits.

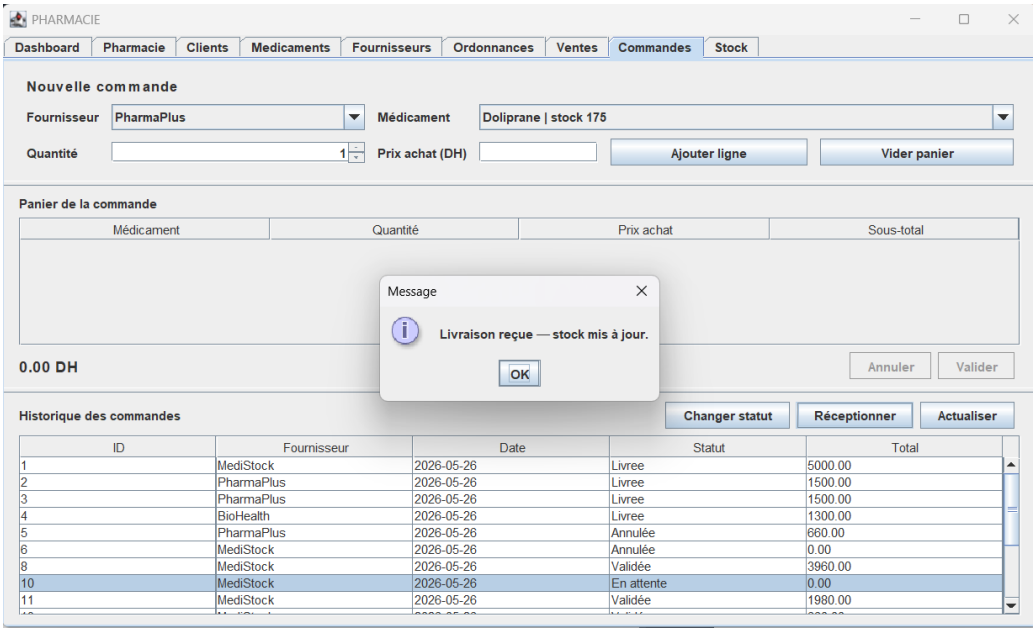


FIGURE 8.14 – Stock réapprovisionné après réception de la commande

8.6 Résultats obtenus

Les différents tests réalisés ont montré que les fonctionnalités principales de l'application fonctionnent correctement. Les données sont enregistrées de manière cohérente dans la base de données et les interactions entre les différents modules sont conformes aux attentes

Chapitre 9 : Limites et perspectives

L'application remplit les objectifs fixés par le sujet, mais plusieurs améliorations sont envisageables pour la rendre plus complète et plus robuste :

1. **Graphiques pour le dashboard** : intégrer des histogrammes et des courbes (via une bibliothèque comme JFreeChart) pour visualiser l'évolution des ventes et du stock, plutôt que de s'appuyer uniquement sur des tableaux textuels.
2. **Export PDF des factures** : permettre l'exportation des factures en format PDF pour l'impression et l'archivage professionnel, à l'aide d'une bibliothèque comme iText.
3. **Authentification** : ajouter un écran de connexion avec un système de mots de passe pour restreindre l'accès à l'application et protéger les données sensibles.
4. **Statistiques graphiques avancées** : proposer des analyses plus fines, comme l'évolution mensuelle du chiffre d'affaires ou les tendances de consommation par catégorie de médicament.

Chapitre 10 : Conclusion

Ce projet nous a permis de concevoir et de développer une application complète de gestion de pharmacie en Java. Le système couvre les opérations principales demandées dans le sujet : gestion des médicaments, suivi du stock avec alertes, gestion des clients, saisie des ordonnances, ventes avec ou sans prescription, commandes fournisseurs avec réception et mise à jour du stock, génération automatique de factures texte et analyse des ventes via un dashboard dédié.

L'architecture en couches que nous avons adoptée Vue, Contrôleur, DAO, Modèle structure le projet de manière lisible et maintenable. Les vues gèrent l'affichage et l'interaction avec l'utilisateur, les contrôleurs appliquent les règles métier, les DAO communiquent avec MySQL et les modèles représentent les données du domaine. Cette séparation nous a permis de travailler en parallèle sur différentes parties du code sans créer de conflits.

Les difficultés rencontrées au fil du développement incohérences entre noms SQL et DAO, gestion de l'ordonnance dans le processus de vente, création de la vente avant l'ajout des lignes, à lire les messages d'exception et à corriger le code de façon méthodique. L'ajout du module Dashboard et de la génération de factures a enrichi notre compréhension de l'intégration entre modules.

Sur le plan des compétences, ce projet nous a permis de consolider nos acquis en Java, Swing, JDBC et MySQL. Nous avons compris concrètement comment structurer un projet en couches, comment connecter une application desktop à une base de données relationnelle, et comment gérer les cas particuliers qui émergent lors de l'intégration de modules interdépendants.

L'application constitue une base solide, fonctionnelle et cohérente, qui pourrait être enrichie par de l'authentification, de l'export PDF, des graphiques et des tests automatisés pour répondre à des besoins plus avancés dans un contexte professionnel.